Hafta 6 — Derste Yapılacak Uygulamalar

> **Toplam süre:** 31 dakika — blok sürelerinin **içindedir** (bkz. `H06-anlatim.md` bölüm 2)
>
> Örnek proje **iki hâlde** gelir: `Baslangic/` (boşluklu) ve `Bitmis/` (tamamlanmış). Derste **Başlangıç'taki boşlukları canlı doldurursunuz**; öğrenci aynı boşlukları evde kendi bilgisayarında doldurur.
>
> Bu haftanın iki aracı: formu **boş ve hatalı** göndermek, ve F12 → **Network** sekmesinde isteğin **gidip gitmediğine** bakmak.

Doldurulacak yerler

Görev	Ne yapılacak	Dosya	Doldurunca ne görülür
TODO 6.1	Başlık ve Yazar için doğrulama attribute'lerini ekleyin: `[Required]` ve `[StringLength]`, Türkçe `ErrorMessage` ile	`ViewModels/KitapEkleVM.cs`	Hafta 6 ders notu bölüm 2.2 — `{0}` alanın adı, `{1}` üst sınır, `{2}` alt sınırdır. Boş gönderin; `/Kitap/EkleTH`'deki hata listesinde mesajınız görünmeli
TODO 6.2	Fiyat ve kategori için `[Required]` ve `[Range]` ekleyin	`ViewModels/KitapEkleVM.cs`	Hafta 6 ders notu bölüm 2.2 — `decimal?` için `[Range(1.0, 10000.0, …)]`; kategori 1, 2 ya da 3 olmalı. TODO'dan önce boş fiyatla gönderin: 0 TL'lik kitap eklenir
TODO 6.3	Hataları gösterin: formun üstüne `asp-validation-summary`, her alanın altına `asp-validation-for`	`Views/Kitap/EkleTH.cshtml`	Hafta 6 ders notu bölüm 3.3 — elle yazılmış ModelState döngüsünün yerini iki Tag Helper alır. `ModelOnly` yalnızca alana bağlı olmayan hataları listeler
TODO 6.4	İş kuralı: aynı başlıkta bir kitap varsa `ModelState.AddModelError` ile hata ekleyin	`Controllers/KitapController.cs`	Hafta 6 ders notu bölüm 3.4 — anahtar `nameof(KitapEkleVM.Baslik)` olursa hata Başlık alanının altına düşer. "Suç ve Ceza" ekleyerek deneyin
TODO 6.5	İstemci doğrulamasını açın: `_ValidationScriptsPartial`'ı sayfaya ekleyin	`Views/Kitap/EkleTH.cshtml`	Hafta 6 ders notu bölüm 4.2 — tek satır. Sonra formu boş gönderin: istek sunucuya GİTMEDEN hata görünmeli (F12 → Network boş kalır)
TODO 6.6	`IsValid` gövdesini yazın: metin büyük harfle başlamıyorsa `false`	`Validation/BuyukHarfleBaslarAttribute.cs`	Hafta 6 ders notu bölüm 5.2 — boş değer bu attribute'ün işi DEĞİL (`[Required]`'ın işi), boşsa `true` dönün. `char.IsUpper`

TODO 6.7	Başlık ile yazar aynı olamaz: `Validate` gövdesini yazın	`ViewModels/KitapEkleVM.cs`	Hafta 6 ders notu bölüm 5.3 — `yield return new ValidationResult("…", [nameof(Baslik), nameof(Yazar)])`. Büyük-küçük harf farkını yok sayın

---

🔧 Hazırlık — Derse girmeden önce (5 dk, ders dışı)

- ☐ `Hafta-06/Baslangic` açıldı, `dotnet run` ile **5072**'de çalıştığı doğrulandı
- ☐ `Hafta-06/Bitmis` **ayrı bir terminalde** açıldı, **5073**'te çalışıyor — takılırsanız yedek
- ☐ Başlangıç'ın **temiz** olduğu doğrulandı: `TODO 6.` aramasında 7 görev, hiçbiri doldurulmamış
- ☐ Editörde dört sekme hazır: `KitapEkleVM.cs`, `EkleTH.cshtml`, `KitapController.cs`, `BuyukHarfleBaslarAttribute.cs`
- ☐ Tarayıcıda F12 → **Network** sekmesi açık; "Preserve log" işaretli
- ☐ `/Dogrula`, `/Dogrula/Ortuk`, `/Dogrula/Tuzak`, `/Dogrula/Sira` bir kez açıldı
- ☐ Tarayıcı yakınlaştırması **125–150%**
- ☐ `H06-etkilesimli.html` açıldı; N denendi ve **kapatıldı**

> 💡 **Ders sonrası:** Başlangıç'ta doldurduklarınızı geri alın ya da klasörü yeniden indirin — bir sonraki sınıfa boş hâliyle girmeli. Eklenen kitaplar bellekte tutulur; uygulama yeniden başlayınca liste sıfırlanır.

Uygulama A — Kuralları yazmak

Süre: 6 dakika · **Blok:** A · **Görevler:** TODO 6.1, 6.2

Amaç: Kural yokken neyin kabul edildiğini görmek, sonra kuralı modele yazıp **aynı** gönderimin değiştiğini göstermek.

Adım 1 — Kuralsız, boş gönderim (1 dk)

`/Kitap/EkleTH`, bütün alanlar boş → **Kaydet**.

Beklenen: form geri gelir; üstteki elle yazılmış listede iki satır:

```
Yazar:  The Yazar field is required.
Baslik: The Başlık field is required.
```

Sınıfa sorulacak: Hiç `[Required]` yazmadık. Bu mesajlar nereden geldi? → `string` soru işaretsiz: çatı onu `[Required]` yazılmış gibi denetliyor (ders notu bölüm 1.3). Mesaj İngilizce, çünkü onu biz yazmadık.

Adım 2 — Başlık ve yazar dolu, fiyat boş (1 dk)

Başlık `deneme`, yazar `deneme`, fiyat ve kategori boş → **Kaydet**.

Beklenen: **302**, listeye **0 TL**'lik bir kitap eklenir; ayrıntısında kategori **Diğer**.

Sınıfa sorulacak: Neden kabul edildi? → `Fiyat` bu hafta `decimal?`: boş kutu `null` olarak bağlandı ve hiçbir kural `null`'u yasaklamıyor.

Adım 3 — TODO 6.1: başlık ve yazar (2 dk)

```
[Required(ErrorMessage = "{0} boş bırakılamaz.")]
[StringLength(100, MinimumLength = 2, ErrorMessage = "{0} {2} ile {1} karakter arasında olmalıdır.")]
[BuyukHarfleBaslar]
[Display(Name = "Başlık")]
public string Baslik { get; set; } = "";
```

Yazar için `[Required]` ve `[StringLength(60, …)]`. Boş gönderin: **Başlık boş bırakılamaz.** Başlığa tek harf yazın: **Başlık 2 ile 100 karakter arasında olmalıdır.**

> ⚠️ **Sık hata:** `{1}` ile `{2}`'yi karıştırmak. `[StringLength]`'te `{1}` üst sınırdır (ilk parametre), `{2}` `MinimumLength`'tir.

Adım 4 — TODO 6.2: fiyat ve kategori (2 dk)

```
[Required(ErrorMessage = "{0} boş bırakılamaz.")]
[Range(1.0, 10000.0, ErrorMessage = "{0} {1} ile {2} arasında olmalıdır.")]
public decimal? Fiyat { get; set; }

[Required(ErrorMessage = "Bir kategori seçin.")]
[Range(1, 3, ErrorMessage = "Geçerli bir kategori seçin.")]
public int? KategoriId { get; set; }
```

Adım 2'yi tekrarlayın: artık **Fiyat (TL) boş bırakılamaz.** ve **Bir kategori seçin.**

Sınıfa sorulacak: Tip `decimal` (soru işaretsiz) olsaydı? → Boş metin bağlamada düşerdi; bizim mesajımız değil **The value '' is invalid.** gelirdi (ders notu bölüm 2.3).

> ⚠️ **Yedek plan:** Proje derlenmiyorsa (`using` eksikliği vb.) Bitmiş'te `✓ TODO 6.1` ve `✓ TODO 6.2`'yi açıp birlikte okuyun.

Uygulama B — Hataları göstermek ve Klasik Tuzak

Süre: 9 dakika · **Blok:** B · **Görevler:** TODO 6.3, 6.4

Amaç: Elle yazılmış `ModelState` döngüsünü Tag Helper'larla değiştirmek, iş kuralı eklemek ve `IsValid` kontrolünün **neyi engellediğini** canlı göstermek.

Adım 1 — TODO 6.3: özet ve alan mesajları (3 dk)

`EkleTH.cshtml`'de üstteki `@if (!ViewData.ModelState.IsValid) { … }` döngüsünü silin, yerine:

```
<div asp-validation-summary="ModelOnly" class="hata-ozet"></div>
```

Her alanın altına:

```
<br /><span asp-validation-for="Baslik" class="hata"></span>
```

Boş gönderin: mesajlar alanların **altında**, kutular **kırmızı çerçeveli**. Ctrl+U: `class="input-validation-error"` ve `class="field-validation-error"` — bu sınıf adlarını çatı yazıyor.

Sınıfa sorulacak: Özet neden boş? → `ModelOnly` yalnızca bir alana bağlı **olmayan** hataları listeler; şu an bütün hatalar bir alana bağlı.

Adım 2 — TODO 6.4: aynı başlık (2 dk)

`KitapController.EkleTH`'de, `IsValid` kontrolünden **önce**:

```
if (KitapDeposu.Hepsi.Any(k => string.Equals(k.Baslik, girdi.Baslik?.Trim(),
        StringComparison.CurrentCultureIgnoreCase)))
    ModelState.AddModelError(nameof(KitapEkleVM.Baslik), "Bu başlıkta bir kitap zaten var.");
```

`Suç ve Ceza` ekleyin (listede var): **Bu başlıkta bir kitap zaten var.** Başlığın altında.

Adım 3 — 🕳 `IsValid`'i silin (3 dk)

`if (!ModelState.IsValid) return View(girdi);` satırını yorum yapın; formu **boş** gönderin.

Beklenen (ölçüldü): **302**, listede başlığı **boş**, fiyatı **0 TL** bir kart.

Sınıfa sorulacak: Kurallar çalıştı mı? → Evet: `ModelState` hatalarla doluydu. Ama hiçbir şey **engellenmedi**: attribute'ler raporlar, engelleyen `if` satırıdır.

Adım 4 — Geri alın (1 dk)

Satırı geri getirin ve boş formu bir kez daha gönderin: **200**, aynı form, hatalarla.

> ⚠️ **Yedek plan:** Yeniden derleme uzun sürerse Adım 3 atlanır; slayttaki ölçülmüş sonuç okunur. Ders notunda Klasik Tuzak bölümünde de var.

Uygulama C — İstemci doğrulaması ve atlatma

Süre: 8 dakika · **Blok:** C · **Görev:** TODO 6.5

Amaç: İstemci doğrulamasının **hız** kazandırdığını, ama **güvenlik** olmadığını aynı formda göstermek.

Adım 1 — TODO 6.5: betikler (2 dk)

`EkleTH.cshtml`'in sonuna:

```
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Boş gönderin. F12 → Network: **istek yok**; mesajlar anında, sunucununkiyle aynı metinle.

Adım 2 — Konsoldan atlatma (2 dk)

Sayfayı yenileyin, F12 → **Console**:

```
document.forms[0].submit()
```

Beklenen (ölçüldü): Network'te **POST**, yanıt **200**; sunucu dört hatanın dördünü de döndürdü. `form.submit()` hem tarayıcının yerleşik denetimini hem jQuery'yi atlar.

> ⚠️ **Sık hata:** F12'de `data-val` attribute'lerini silmek. İşe **yaramaz** — kurallar sayfa açılırken bir kez kuruldu (ölçüldü). Bu yüzden konsol satırı kullanılır.

Adım 3 — Ctrl+U (2 dk)

`Baslik` alanında `data-val-required="Başlık boş bırakılamaz."`, `data-val-length`, `maxlength="100"`; sayfanın sonunda üç betik: `jquery.validate`, `…unobtrusive`, `dogrulama-tr.js`.

Sınıfa sorulacak: Türkçe harfler neden `Başlık` gibi görünüyor? → Razor çıktıyı kodluyor (Hafta 3 detay); tarayıcı doğru gösterir.

Adım 4 — Düzeltmesiz hâl (2 dk)

`/Dogrula?istemci=true&ham=true` → **Boy** alanına `1,75`: **must be a number**. `1.75` yazın: istemci geçirir, sunucu **175** okur ve "between 0,5 and 2,5" der. `?istemci=true` (düzeltmeli) ile aynı `1,75` geçer.

> ⚠️ **Yedek plan:** Konsol kullanmak istemezseniz F12 → ⚙ → **Disable JavaScript**, sayfayı yenileyip gönderin: sonuç aynı (ölçüldü).

Uygulama D — Kendi kurallarımız

Süre: 8 dakika · **Blok:** C · **Görevler:** TODO 6.6, 6.7

Amaç: Kendi `ValidationAttribute`'ünü ve `IValidatableObject`'i yazmak; ikisinin de **yalnızca sunucuda** çalıştığını ve `Validate`'in **sırasını** göstermek.

Adım 1 — TODO 6.6: `IsValid` (3 dk)

```
public override bool IsValid(object? value)
{
    if (value is not string metin || metin.Length == 0)
        return true;                  // boşluğu [Required] yakalar
    return char.IsUpper(metin[0]);
}
```

Başlık `küçük harfli`, diğerleri doğru → **Kaydet**. Beklenen (ölçüldü): istemci **geçirdi** (Network'te POST), sunucu **Başlık büyük harfle başlamalıdır.** ile geri çevirdi.

Adım 2 — Ctrl+U (1 dk)

`Baslik` alanının `data-val-*` listesi: `required` ve `length` var, **kendi kuralımız yok**. İstemci onu bilmiyor.

Adım 3 — TODO 6.7: `Validate` (3 dk)

```
public IEnumerable<ValidationResult> Validate(ValidationContext validationContext)
{
    if (string.Equals((Baslik ?? "").Trim(), (Yazar ?? "").Trim(),
            StringComparison.CurrentCultureIgnoreCase))
        yield return new ValidationResult("Başlık ile yazar aynı olamaz.",
            [nameof(Baslik), nameof(Yazar)]);
}
```

Başlık `Orhan Pamuk`, yazar `orhan pamuk`, fiyat `120,50`, kategori seçili → **Başlık ile yazar aynı olamaz.** iki alanın da altında.

Adım 4 — Sırayı görmek (1 dk)

Aynısını fiyat `0` ile, **konsoldan** gönderin:

```
document.forms[0].submit()
```

Beklenen (ölçüldü): **200**, yalnızca **Fiyat (TL) 1 ile 10000 arasında olmalıdır.**; "aynı olamaz" **yok**. Çatı `Validate`'i yalnızca özellik kuralları geçince çağırır.

Sınıfa sorulacak: Neden **Kaydet** düğmesiyle değil de konsoldan? → Düğmeyle gönderirseniz istemci fiyatı yakalar ve istek hiç gitmez; sunucudaki sırayı göremezsiniz (ölçüldü).

> ⚠️ **Sık hata:** `Validate` içinde `Baslik.Trim()` yazmak. Özellik kuralları geçmeden çağrılmaz ama sırayı bilmeyen biri `null` kontrolünü atlarsa ileride `NullReferenceException` alır. `(Baslik ?? "")` alışkanlık olsun.

> ⚠️ **Yedek plan:** Süre kalmadıysa `/Dogrula/Sira` sayfasındaki iki düğme aynı sırayı gösterir; TODO 6.7 ev görevine kalır.

🏠 Ders Sonrası Öğrenci Görevi

0. **Başlangıç** projesinde **TODO 6.1 – 6.7**'yi tamamlayın.
1. Kendi projenizdeki kayıt formunun ViewModel'ine **en az beş** doğrulama kuralı yazın; her birine Türkçe `ErrorMessage` verin. Değer tiplerinde nullable kuralını unutmayın.
2. POST action'ınızı kanonik kalıpla yazın: geçersizse `View(girdi)`, geçerliyse kaydedip `RedirectToAction`.
3. Formda her alanın altına `asp-validation-for`, üstüne `asp-validation-summary` koyun; `_ValidationScriptsPartial`'ı ekleyin.
4. Sunucunun gerçekten denetlediğini **kanıtlayın**: istemci doğrulaması açıkken konsoldan `document.forms[0].submit()` ile boş formu gönderin; sunucunun döndürdüğü hataların ekran görüntüsünü alın.
5. Birden çok alana bakan bir kural için `IValidatableObject` yazın.

Teslim: proje klasörü + 4. maddenin ekran görüntüsü + şu soruya **iki cümlelik** cevap: *"İstemci doğrulaması varken sunucu doğrulamasına neden hâlâ ihtiyaç var?"*

✅ Hocaya Kontrol Listesi

- ☐ Network sekmesinin yazı tipi **arka sıradan okunacak** kadar büyük
- ☐ Başlangıç (5072) ve Bitmiş (5073) **iki ayrı sekmede**, geçiş hızlı
- ☐ Uygulama B Adım 4 (`IsValid`'i geri alma) **atlanmadı**
- ☐ Uygulama C Adım 2'de **konsol** satırı kullanıldı, `data-val` silme denenmedi
- ☐ Uygulama D Adım 1'de istemci doğrulaması **açıktı** (TODO 6.5 yapılmış)
- ☐ Ders sonunda Başlangıç **boş hâline döndürüldü**

⚠️ Yedek Planların Özeti

Sorun	Yedek
Bir boşluk derste çalışmıyor	Sağ sekmede **Bitmiş** (5073); dosyada `✓ TODO 6.x` aratın
Konsol kullanılamıyor	F12 → ⚙ → **Disable JavaScript**, yenile, gönder
`dotnet run` yeniden başlatma yavaş	Uygulama B Adım 3 atlanır; slayttaki ölçülmüş sonuç okunur
Network sekmesi projeksiyonda okunmuyor	`/Dogrula` sayfasındaki `ModelState` tablosu ve durum kodu satırı kullanılır
Proje hiç açılmıyor	Bütün çıktılar ders notunda tablo hâlinde var; slaytlardan anlatılır